To be designed in python:

You're a senior developer and low-fantasy/worldbuilding writer. Design a comprehensive, production-grade worldbuilding application for fiction writers—especially writers who benefit from externalizing complex spatial, genealogical, political, economic, historical, and interpersonal information rather than keeping it mentally visualized.

The application should function as a unified **fictional world modeling, visualization, continuity, and writing-support system**. It should go far beyond a wiki, family-tree maker, map editor, note-taking tool, or character database.

The central idea is:

The fictional world should exist as a structured, interconnected model that the writer can inspect from many different perspectives.

A writer should be able to model people, families, noble houses, governments, settlements, regions, geography, roads, waterways, trade, resources, agriculture, economies, military forces, religions, cultures, languages, historical events, political conflicts, secrets, character knowledge, relationships, succession, travel, logistics, and story scenes—and have all of these systems interact.

The software should act as an **external cognitive model of the fictional world**.

Do not design this as a shallow CRUD application with disconnected forms. Design it as an interconnected world graph with temporal state, geographic state, causality, rule systems, validation, derived information, visualization, and narrative tooling.

1. Product vision

Create a desktop-first but responsive worldbuilding platform intended for writers of:

- low fantasy
- historical fantasy
- political fantasy
- dynastic fiction
- epic fantasy
- historical fiction
- mysteries involving large casts
- alternate history
- role-playing campaign worlds
- continuity-heavy fictional universes

The software should support both writers who prefer simple worldbuilding and writers who want extremely deep simulation.

It should be possible to begin with:

“There are three towns, two noble houses, and a disputed inheritance.”

…and gradually expand that world into hundreds or thousands of interconnected entities without having to redesign the project.

The application should prioritize:

1. clarity
2. interconnected information
3. visual comprehension
4. temporal consistency
5. narrative usefulness
6. discoverability
7. extensibility
8. writer control

The goal is not to force the writer to simulate everything.

The goal is to let the writer model **exactly as much as they need** while preserving relationships between systems.

2. Core conceptual architecture

Base the world model around several fundamental concepts.

Entity

Anything that exists as an identifiable thing.

Examples:

- person
- family
- dynasty
- noble house
- clan
- government
- kingdom
- empire

- region
- province
- county
- duchy
- settlement
- village
- town
- city
- castle
- manor
- monastery
- port
- mine
- forest
- mountain
- river
- lake
- sea
- road
- bridge
- trade route
- religious organization
- military organization
- guild
- culture
- ethnic group
- language
- resource
- commodity
- artifact
- law
- title
- office
- army
- fleet
- historical document
- event
- war
- battle
- treaty
- scene
- story chapter

Entities should support customizable schemas.

Users must be able to create their own entity types.

Relationship

Relationships connect entities.

Examples:

- parent of
- child of
- sibling of
- married to
- betrothed to
- lover of
- friend of
- rival of
- distrusts
- hates
- protects
- owes debt to
- patron of
- serves
- vassal of
- sworn to
- allied with
- at war with
- trades with
- controls
- legally owns
- occupies
- claims
- administers
- taxes
- produces
- imports
- exports
- contains
- located in
- borders
- connected by
- flows through
- worships
- belongs to
- speaks

- inherited from
- killed
- participated in
- witnessed
- knows about

Relationships should support:

- directionality
- bidirectionality
- start date
- end date
- strength
- status
- legality
- secrecy
- confidence
- notes
- tags
- custom properties
- source/evidence
- visibility
- historical changes

A relationship should be able to be true during one historical period and false during another.

3. Temporal world model

Time should be a first-class system rather than a text field.

Support:

- absolute dates
- relative dates
- custom calendars
- eras
- seasons
- uncertain dates
- date ranges
- approximate dates
- recurring events

Allow users to create fictional calendars with:

- custom months
- custom month lengths
- leap rules
- week lengths
- named days
- eras
- holidays
- seasons

Every relevant entity property or relationship should optionally have a temporal range.

Example:

House Marr controls Greyhaven:

- start: 312
- end: 428

After 428:

House Orren controls Greyhaven.

The system should therefore support a **world-state-at-date** query.

A global timeline slider should let the writer select:

Year 215

and have maps, titles, political control, borders, alliances, marriages, living characters, rulers, wars, settlement ownership, and other temporal information update accordingly.

4. Character system

Characters should support much more than ordinary profiles.

Include:

Identity

- full name
- aliases
- titles
- honorifics
- nickname

- gender
- age
- birth date
- death date
- birth location
- current location
- culture
- nationality
- ethnicity
- religion
- languages
- social rank
- occupation

Physical description

- height
- build
- hair
- eyes
- distinguishing features
- injuries
- disabilities
- clothing style
- equipment
- portrait/reference images

Allow entirely optional visual fields.

Psychology

- personality traits
- values
- worldview
- fears
- desires
- prejudices
- loyalties
- ambitions
- insecurities
- coping mechanisms
- habits
- moral boundaries

Motivation model

Every important character should optionally support:

Surface goal

What they openly claim to want.

Private goal

What they actually want.

Fundamental need

What emotionally drives them.

Fear

What they most want to avoid.

Pressure point

What circumstance might cause them to abandon normal behavior.

Stakes

What they lose if they fail.

Current obstacle

What prevents success.

Resources

What they can use.

Leverage

What they can use against other people.

Vulnerabilities

What can be used against them.

5. Relationship psychology

Relationships should not simply be labels such as “friend” or “enemy.”

Support asymmetric relationships.

Example:

Mara → trusts → Edric

Edric → distrusts → Mara

Optional dimensions could include:

- affection
- trust
- fear
- respect
- resentment
- dependency
- attraction
- loyalty
- suspicion
- obligation
- perceived power imbalance

Allow qualitative states rather than requiring numbers.

Examples:

- deeply trusts
- somewhat trusts
- uncertain
- suspicious
- actively hostile

Users should be able to define their own scales.

6. Knowledge, belief, secrets, and misinformation

Implement separate layers for:

Objective truth

What is actually true in the world.

Character knowledge

What a character knows.

Character belief

What a character believes.

Character suspicion

What they suspect but cannot prove.

Misinformation

What they incorrectly believe.

Secret ownership

Who knows a secret.

Secret awareness

Who knows that another person knows.

Example:

Objective truth:

Prince Oren is illegitimate.

Mara:

knows Oren is illegitimate.

Oren:

believes he is legitimate.

Tomas:

suspects Oren is illegitimate.

Sera:

knows Mara knows.

This should allow deeply layered dramatic irony.

The writer should be able to query:

Who knows that Prince Oren is illegitimate?

and separately:

Who believes Prince Oren is illegitimate?

7. Genealogy and dynasties

Create an advanced genealogy system supporting:

- biological parents
- legal parents
- adoptive parents
- foster relationships
- disputed parentage
- illegitimate children
- legitimized children
- secret parentage
- uncertain parentage
- marriages
- remarriages
- divorces
- annulments
- betrothals
- concubinage
- sibling relationships
- half siblings
- step families
- cadet branches
- dynasty changes
- house membership changes

Visualize large dynastic trees cleanly.

Allow filters such as:

- living only
- specific generation
- descendants
- ancestors
- members of a house
- eligible heirs
- disputed lineage

8. Titles and inheritance

Titles should be separate entities.

Examples:

- King of Renn
- Duke of Northwatch
- Lord of Greyhaven
- High Priest of Aramor
- Warden of the Western March

Titles should support:

- rank
- legal jurisdiction
- territorial association
- ceremonial importance
- current holder
- historical holders
- creation date
- extinction date
- succession law
- appointment method

Implement inheritance and succession systems.

Support common systems such as:

- absolute primogeniture
- male-preference primogeniture
- male-only primogeniture

- ultimogeniture
- seniority
- elective monarchy
- tanistry
- appointment
- conquest
- hereditary office

Allow custom rule scripting.

The software should calculate succession order.

Example:

Death of King Aldren

Current succession:

1. Prince Oren
2. Lady Elia
3. Lord Caros
4. Lady Mara

If Oren is declared illegitimate:

1. Lady Elia
2. Lord Caros
3. Lady Mara

Allow hypothetical succession calculations without modifying canonical history.

9. Noble houses and political families

Noble houses should support:

- founder
- founding date
- current head
- seat
- ancestral lands
- titles
- heraldry
- motto
- cadet branches

- family tree
- political alignment
- wealth
- prestige
- military strength
- economic assets
- allies
- rivals
- vassals
- lieges
- claims
- debts
- marriages
- historical feuds
- cultural identity
- religious identity

Distinguish:

- dynasty
- house
- branch
- clan
- lineage
- political faction

10. Feudal hierarchy

The system should model hierarchical obligation without assuming every world uses European feudalism.

Support customizable political structures.

For feudal-like worlds, support:

- sovereign
- great houses
- dukes
- counts
- barons
- landed knights
- minor houses
- unlanded nobility

- clergy
- chartered towns

Represent:

House Marr → vassal of → House Veyne

House Veyne → vassal of → Crown

But allow exceptions, disputed allegiances, dual loyalties, and personal unions.

11. Territorial ownership and control

Critically, distinguish:

- legal owner
- nominal ruler
- administrative controller
- military occupier
- tax authority
- religious authority
- economic controller
- claimant
- de facto local ruler

A region could therefore be:

Legally owned by House Orren.

Administered by House Veyne.

Occupied by House Marr.

Taxed by the Crown.

Claimed by two rival heirs.

This distinction should be visible on maps and entity pages.

12. Geography system

Create a layered geographic model.

Potential hierarchy:

World
→ continent
→ realm
→ geographic region
→ political region
→ province
→ county
→ holding
→ settlement
→ site

Do not force one hierarchy.

Physical geography should support:

- elevation
- mountains
- hills
- valleys
- plains
- wetlands
- forests
- deserts
- tundra
- coastlines
- islands
- rivers
- lakes
- watersheds
- rainfall
- temperature
- soil
- vegetation
- climate zones
- natural hazards

Political boundaries and geographic boundaries should be separate.

13. Regional modeling

Each region should optionally track:

- area
- population
- population density
- urbanization
- terrain
- climate
- soil fertility
- rainfall
- water availability
- agricultural potential
- timber
- mineral resources
- grazing
- fisheries
- ports
- roads
- navigable rivers
- strategic value
- defensibility
- trade access
- tax value
- cultural composition
- religious composition
- ruling house
- local houses
- local disputes

14. Regional advantages and disadvantages

Allow writers to manually specify advantages/disadvantages.

Also derive possible implications from underlying data.

Example:

Region:

- mountainous
- iron-rich
- forested
- low agricultural capacity

- one major mountain pass

Derived possibilities:

Advantages:

- defensible
- mining economy
- timber production
- strategic pass
- natural fortification

Disadvantages:

- grain dependence
- costly road construction
- winter isolation
- limited population
- expensive transportation

Derived information should always be presented as suggestions rather than unquestionable truth unless the writer explicitly enables simulation rules.

15. Settlement system

Settlement types should include:

- hamlet
- village
- town
- city
- capital
- castle settlement
- mining settlement
- monastery
- fortress
- port
- market town
- pilgrimage center

Settlement properties may include:

- population
- founding date

- ruling authority
- controlling house
- local government
- tax obligations
- fortifications
- markets
- industries
- guilds
- imports
- exports
- agricultural hinterland
- water source
- sanitation
- roads
- river access
- port access
- nearby resources
- food supply
- wealth
- crime
- stability
- population growth
- cultural groups
- religions
- landmarks

16. Rural holdings

Model economically important holdings such as:

- manors
- farms
- estates
- mills
- mines
- quarries
- vineyards
- forests
- hunting grounds
- fisheries
- pasture
- salt works
- workshops

Rights should be independently modelable:

- land ownership
- grazing rights
- fishing rights
- timber rights
- mineral rights
- hunting rights
- toll rights
- bridge rights
- market rights
- taxation rights

17. Resources and commodities

Implement a flexible resource system.

Categories might include:

Agriculture

- wheat
- rye
- barley
- oats
- rice
- vegetables
- fruit
- grapes
- flax

Livestock

- cattle
- sheep
- pigs
- goats
- horses
- poultry

Natural resources

- timber
- stone
- clay
- salt
- coal
- iron
- copper
- tin
- silver
- gold

Manufactured goods

- textiles
- tools
- weapons
- armor
- pottery
- glass
- paper
- wine
- beer
- ships

Fantasy resources

Users can create anything.

Each resource can include:

- production requirements
- terrain requirements
- climate requirements
- labor
- technology requirements
- production regions
- consumers
- rarity
- strategic importance
- price category
- perishability
- transport difficulty

18. Production system

Support multiple levels of economic detail.

Simple mode

Examples:

- grain production: high
- iron production: medium
- wine production: none

Intermediate mode

Relative production units.

Advanced mode

Estimated quantities.

Do not require advanced economics.

Allow users to choose complexity per project or system.

Production can be influenced by:

- available land
- soil fertility
- climate
- water
- labor
- infrastructure
- technology
- war
- disease
- weather
- taxation
- political stability

19. Trade system

Trade should be represented as a network.

Model:

- origin
- destination
- commodity
- route
- volume
- season
- tolls
- transport method
- travel time
- risk
- profitability
- political control

Trade routes may use:

- roads
- rivers
- canals
- coastal shipping
- sea lanes
- mountain passes

The writer should be able to view:

Where does Greyhaven get its grain?

and trace the supply path.

20. Roads and infrastructure

Roads should be first-class entities.

Properties:

- name
- start
- end
- intermediate locations
- length
- terrain

- width
- quality
- construction type
- maintenance authority
- toll authority
- bridges
- ferries
- passes
- seasonal closures
- military importance
- trade importance
- danger
- average travel speed

Infrastructure may include:

- bridges
- ferries
- canals
- aqueducts
- ports
- docks
- warehouses
- watchtowers
- roadside inns
- checkpoints
- walls

21. Water system

Model:

- river
- stream
- lake
- sea
- bay
- marsh
- canal
- spring
- aquifer

For rivers, include:

- source
- mouth
- tributaries
- flow direction
- navigation
- flood patterns
- bridges
- ferries
- ports
- mills
- irrigation
- drinking-water dependency
- political boundaries

Water should influence settlements and trade.

22. Travel and logistics

Implement route calculation.

The writer should be able to ask:

How long does it take to travel from Greyhaven to Rennford?

Factors:

- distance
- road quality
- terrain
- weather
- season
- transport type
- horse availability
- river transport
- military baggage
- party size
- elevation
- border crossings
- ferries

Support:

- walking
- horse

- carriage
- wagon
- river barge
- sailing ship
- army
- messenger

Allow configurable assumptions.

23. Military system

Model military power without requiring a strategy-game simulation.

Potential entities:

- army
- fleet
- regiment
- levy
- household guard
- mercenary company
- garrison

Properties:

- size
- composition
- commander
- allegiance
- recruitment source
- readiness
- experience
- equipment
- supply
- morale
- location

Political entities may have estimated:

- levy capacity
- standing forces
- naval strength
- fortifications

24. Military geography

Maps should identify:

- castles
- fortresses
- walls
- bridges
- mountain passes
- ports
- chokepoints
- supply corridors
- navigable rivers
- defensible terrain

Queries should include:

Which route would an invading army most likely use?

Which bridge is strategically essential?

Which settlements would need to be captured before reaching the capital?

25. Politics and factions

Support organizations such as:

- royal court
- noble faction
- merchant guild
- clergy faction
- military faction
- rebel movement
- secret society
- council

Track:

- members
- leadership
- goals
- ideology
- resources

- influence
- allies
- enemies
- internal factions
- territory
- historical activity

26. Government and law

Allow customizable government systems.

Examples:

- monarchy
- elective monarchy
- aristocratic republic
- theocracy
- tribal confederation
- city-state
- imperial bureaucracy

Track:

- offices
- laws
- legal rights
- taxation
- jurisdiction
- courts
- punishments
- administrative divisions
- succession
- political institutions

27. Religion

Religion should be modeled as more than a text description.

Include:

- gods
- doctrine

- cosmology
- rituals
- clergy
- hierarchy
- temples
- monasteries
- sacred locations
- festivals
- taboos
- denominations
- heresies
- holy texts
- political influence

Model religious geography.

Allow religious membership to vary by:

- person
- settlement
- region
- house
- culture

28. Culture

Cultures may include:

- naming conventions
- customs
- clothing
- food
- architecture
- marriage practices
- inheritance norms
- attitudes toward magic
- social expectations
- funeral customs
- gender roles
- hospitality
- class structure
- festivals
- warfare traditions

Cultures should have geographic and historical distribution.

29. Languages

Track:

- language
- dialect
- script
- geographic distribution
- cultural association
- prestige
- mutual intelligibility
- historical changes

Characters should have varying proficiency.

30. Magic

Magic systems should be optional and highly configurable.

Support:

- magic type
- source
- rules
- costs
- limitations
- known practitioners
- institutions
- cultural beliefs
- legal restrictions
- magical resources
- historical magical events

For low fantasy, the system should work just as well when magic is rare, ambiguous, or poorly understood.

Distinguish:

what magic objectively does

from

what cultures believe magic does.

31. Historical events

Events should be structured objects.

Examples:

- birth
- death
- marriage
- coronation
- battle
- war
- treaty
- famine
- plague
- rebellion
- migration
- religious schism
- discovery
- assassination
- succession crisis
- natural disaster

Events should support:

- date
- duration
- location
- participants
- witnesses
- cause
- consequence
- casualties
- political effects
- economic effects
- territorial changes
- sources
- conflicting interpretations

32. Causality

Allow events to form causal chains.

Example:

Flood
→ crop failure
→ grain shortage
→ price increase
→ urban unrest
→ tax revolt
→ noble crackdown
→ rebellion

Users should be able to manually define causal relationships.

Optionally suggest likely downstream consequences.

Create a **consequence explorer**.

Selecting an event should show:

- direct effects
- secondary effects
- political effects
- economic effects
- character effects
- geographic effects

33. Historical interpretation

Different people or cultures may interpret the same event differently.

Example:

Battle of Red Ford

Crown version:

heroic victory.

Northern version:

massacre.

Religious version:

divine punishment.

Allow multiple narratives surrounding one objective event.

34. Maps

Mapping should be one of the application’s major interfaces.

Users should be able to import, draw, or create fictional maps.

Support vector geographic objects where possible.

Map elements:

- political boundaries
- physical regions
- settlements
- castles
- rivers
- roads
- trade routes
- forests
- mountains
- resources
- ports
- battles
- religious sites
- cultural areas

35. Map layers

Provide switchable layers such as:

- terrain
- elevation
- climate
- political boundaries
- feudal control

- legal ownership
- military occupation
- population
- settlements
- trade
- agriculture
- resources
- roads
- waterways
- military
- religion
- culture
- language
- historical events

Allow multiple layers simultaneously.

Example:

Show House Veyne holdings + major roads + grain production.

36. Temporal maps

Maps should update according to selected date.

Changing the year may alter:

- borders
- ownership
- ruler
- roads
- settlements
- names
- trade routes
- religious distribution
- wars
- population
- political control

37. Character maps

Selecting a character could display:

- birthplace
- childhood locations
- education
- travel
- marriages
- battles
- residences
- current location

Allow character routes across time.

38. Relationship graph

Create an interactive graph for social and political relationships.

Nodes may include:

- people
- houses
- governments
- organizations
- locations

Edges may represent:

- blood
- marriage
- loyalty
- rivalry
- trade
- debt
- political alliance
- hatred
- secrecy
- military obligation

Allow filtering by relationship type.

39. Genealogy visualization

Provide a specialized pedigree view separate from the generic relationship graph.

Support very large families.

Features:

- collapse branches
- highlight descendants
- highlight ancestors
- show inheritance
- indicate legitimacy
- indicate uncertain parentage
- show house membership
- show marriages
- show title succession

40. Political hierarchy visualization

Create visual trees showing:

Crown
↓
Great House
↓
Major vassal
↓
Minor house
↓
landed knight

Allow overlays for:

- rebellion
- secret allegiance
- disputed loyalty
- military occupation

41. Trade visualization

Trade map should show:

- routes
- direction
- commodity

- relative volume
- strategic dependency

Selecting a commodity should show:

- producers
- consumers
- trade routes
- import-dependent locations

42. Economic dependency visualization

Allow queries such as:

Which towns depend on imported grain?

Which regions depend on Greyhaven’s harbor?

What happens if this river becomes unnavigable?

43. Story and manuscript integration

The application should not end at worldbuilding.

Integrate writing-oriented tools.

Entities should be linkable to:

- novels
- books
- acts
- chapters
- scenes
- outlines

44. Scene system

A scene should contain:

- title
- date

- location
- participating characters
- POV character
- objective
- conflict
- outcome
- secrets revealed
- knowledge gained
- relationships changed
- timeline position

When opening a scene, automatically surface relevant world information.

Example:

Scene:

Winter Feast at Greyhaven

Participants:

- Mara
- Tomas
- Edric
- Queen Sera
- Prince Oren

Display:

Relevant relationships

- Mara secretly loves Edric.
- Tomas distrusts Edric.
- Edric owes Sera money.
- Sera suspects Mara of treason.

Relevant secrets

- Mara knows Oren’s parentage.
- Oren does not.
- Sera knows Mara knows.

Active goals

- Mara wants Tomas to support Oren.
- Edric wants Oren discredited.

- Sera wants to identify the leak.

Recent history

- Edric's brother was executed eleven days earlier.
- Mara approved the execution.

The writer should not need to remember these facts manually.

45. Narrative relevance engine

For any scene, rank related facts by relevance.

Potential relevance factors:

- participants
- location
- recent events
- unresolved conflicts
- active goals
- secrets
- family relationships
- political relationships
- geographic conditions
- economic pressures

Avoid overwhelming the writer with every fact in the database.

46. Continuity engine

Implement automated continuity checking.

Examples:

Character appears in two distant locations on the same date.

Character references an event that has not happened yet.

Dead character appears without explanation.

Title is used before it was created.

Marriage predates one participant's birth.

A road is used before construction.

Character knows a secret before learning it.

City belongs to the wrong house on this date.

Journey requires three days but scene timeline allows one.

Continuity violations should have severity levels.

Allow intentional exceptions.

47. Contradiction detection

Detect conflicting facts.

Example:

Person’s death date: 229

Battle participation: 231

Flag:

Elia is listed as participating in the Battle of Orren in 231 but died in 229.

48. Consistency rules

Create customizable rule systems.

Examples:

- characters cannot give birth before configurable minimum age
- succession follows specific law
- title holders must be alive
- settlement must belong to a region
- river routes must follow connected segments
- characters cannot know information before learning it

Rules should be overrideable.

Fantasy worlds should not be forced to follow real-world assumptions.

49. Query system

One of the most important features should be the ability to interrogate the world.

Examples:

Who hates Lord Marek?

Who benefits if Elian dies?

Who knows the king’s secret?

Who is related to Lady Mara within three generations?

Which houses control iron mines?

Which houses serve House Veyne?

Which settlements are controlled by House Marr?

Which regions produce more grain than they consume?

Which towns depend on imported food?

Which ports serve the northern provinces?

What is the fastest route from Greyhaven to the capital?

Which bridges would an invading army need?

Who controls that bridge?

What changed politically between 220 and 240?

Which characters were alive during the Red War?

What secrets are relevant to this scene?

Support both structured filters and natural-language querying.

50. Hypothetical mode

Create a non-canonical sandbox.

Examples:

What happens to succession if Prince Oren dies?

What happens if Greyhaven changes allegiance?

What regions experience food shortages if the River Renn is blockaded?

Which houses gain power if House Marr becomes extinct?

Changes in hypothetical mode must never alter canonical data unless explicitly committed.

51. “Why does this matter?” mode

Allow the writer to select anything and ask:

Why is this important?

For a bridge:

- controls major river crossing
- links grain-producing region to capital
- owned by House Veyne
- produces toll revenue
- only nearby army crossing

For a person:

- third in succession
- heir to two estates
- marriage connects rival houses
- knows royal secret
- commands border army

This feature should trace both authored and derived importance.

52. “What changes if this disappears?” mode

For any entity, calculate dependencies.

Example:

Remove Greyhaven port.

Potential consequences:

- northern exports decline
- grain imports become more expensive
- House Marr loses customs revenue
- alternative trade shifts to Blackmere
- House Orren gains influence

This should be clearly labeled as analytical inference rather than canon unless accepted by the writer.

53. Search

Implement universal search across all entities.

Support:

- exact match
- fuzzy match
- tags
- properties
- relationships
- timeline
- location

Examples:

all dead members of House Veyne

settlements producing wine

characters currently in Greyhaven

54. Views

The same underlying data should support many views.

Required views:

- world dashboard
- entity profile
- map
- timeline

- genealogy
- social graph
- political hierarchy
- succession
- trade network
- resource network
- character knowledge
- scene context
- event causality
- geographic hierarchy
- settlement list
- faction view

55. Workspaces

Allow users to create workspaces focused on tasks.

Examples:

Writing Mode

Current chapter + relevant world information.

Politics Mode

Houses + alliances + succession + territorial control.

Economy Mode

Production + resources + trade.

History Mode

Timeline + events + historical map.

Character Mode

relationships + motivation + secrets.

56. Progressive complexity

The UI must not expose hundreds of fields immediately.

Design progressive complexity.

A beginner might see:

Name
Type
Description
Relationships

An advanced user can expand:

Economics
History
Demographics
Legal status
Temporal properties
Custom rules

Avoid intimidating forms.

57. Data confidence

Facts may be:

- canonical
- tentative
- rumored
- disputed
- intentionally unknown
- author speculation

Allow confidence/status markers.

Example:

Parent of Prince Oren:

King Aldren — publicly believed

Lord Corren — canonical secret

58. Sources and notes

Allow facts to cite internal sources.

Examples:

- chapter
- manuscript scene
- historical document
- author note
- timeline event

Useful for large fictional universes.

59. Version history

Worldbuilding often changes during writing.

Support:

- revision history
- undo/redo
- entity history
- relationship history
- restore points

Potentially support branches:

Canon

Draft rewrite

Alternative history

60. Custom fields and schemas

Users should be able to customize almost everything.

Create:

- custom entity types
- custom relationship types
- custom properties
- custom statuses
- custom categories

- custom inheritance systems
- custom calendars
- custom resource types

Avoid locking the software to European medieval fantasy.

61. Templates

Provide optional templates.

Possible project templates:

- low fantasy kingdom
- dynastic political fantasy
- merchant republic
- nomadic society
- feudal realm
- city-state campaign
- historical-fiction setting

Templates should create schemas, not prewritten lore.

62. Import/export

Support:

- JSON
- CSV
- Markdown
- images
- map formats where practical
- GEDCOM for genealogy if appropriate

Allow full project backup.

Potential manuscript integrations can be considered separately.

63. Offline-first considerations

Strongly consider local-first architecture.

Worldbuilding projects may contain:

- unpublished manuscripts
- proprietary worlds
- private creative work

Privacy should be a major consideration.

Potential capabilities:

- local database
- optional cloud sync
- encrypted backups
- portable project export

64. Technical architecture

Design a technical architecture appropriate for a large interconnected graph-like data model.

Evaluate approaches such as:

- relational database with graph-oriented relationship tables
- graph database
- hybrid relational/graph architecture

Do not choose technology solely because “graph database” sounds appropriate.

Analyze:

- query complexity
- performance
- portability
- offline support
- migrations
- extensibility
- temporal data
- geospatial data
- synchronization

65. Suggested core conceptual schema

At minimum analyze structures resembling:

Entity

- id
- type
- name
- description
- properties
- created_at
- updated_at

Relationship

- id
- source_entity_id
- target_entity_id
- relationship_type
- start_date
- end_date
- properties
- visibility
- certainty

Event

- id
- type
- start_date
- end_date
- location
- participants
- causes
- consequences

Location geometry

- entity_id
- geometry
- geographic_parent
- political_parent

Knowledge state

- observer
- fact
- knowledge_type
- acquired_at

- source

Do not assume this is the final schema. Improve it substantially.

66. Derived properties

Distinguish between:

Authored data

Explicitly entered by the writer.

Derived data

Calculated from authored information.

Examples:

Authored:

Greyhaven imports grain from Rennford.

Derived:

Greyhaven is dependent on the Rennford trade route.

Derived information must show how it was calculated.

67. Explanation transparency

When the software suggests something, provide reasoning.

Example:

Greyhaven may be vulnerable to blockade because 74% of modeled grain imports arrive through its port.

Avoid unexplained black-box conclusions.

68. Simulation philosophy

Do not accidentally turn the application into a mandatory grand-strategy game.

Simulation should be modular.

Each system can be:

- off
- descriptive
- assisted
- simulated

Example:

Economy OFF

Writer enters lore manually.

Economy DESCRIPTIVE

Writer records production levels.

Economy ASSISTED

Software suggests consequences.

Economy SIMULATED

Software estimates supply and demand.

69. Visual accessibility

Because a major use case is helping users externalize complex worlds, visualization quality is essential.

Use:

- clear typography
- color plus non-color indicators
- icons
- labels
- grouping
- collapsible layers
- filtering
- zoom
- focus mode

- breadcrumbs

Never rely solely on color.

Allow users to simplify dense graphs.

70. Aphantasia-oriented UX philosophy

Do not assume users mentally picture scenes or spatial relationships.

Provide external representations.

Examples:

Instead of:

Imagine where the castle lies relative to the river.

Show:

- map
- distance
- elevation
- road connections
- nearby settlements

Instead of:

Remember who dislikes whom.

Show:

- relationship graph
- scene-relevant relationships

Instead of:

Remember family relationships.

Show:

- genealogy

Instead of:

Mentally track political consequences.

Show:

- dependency and causal diagrams

The application should reduce reliance on internal visualization and working memory.

71. World completeness indicators

Optionally help users discover missing details.

Example region:

Terrain ✓
Climate ✓
Settlement hierarchy ✓
Ruling house ✓
Agriculture ✓
Trade ?
Religion ?
Road network incomplete

This must be advisory rather than gamified pressure.

72. Worldbuilding prompts

Contextual prompts can help writers expand areas.

Example:

Settlement has 8,000 inhabitants but no food sources defined.

Ask:

Where does this city obtain its food?

Region has valuable iron but no trade route.

Ask:

How is this iron transported?

Major noble house has no income sources.

Ask:

What supports this house economically?

These prompts should arise from the world model.

73. Plausibility assistant

Optional—not authoritative.

Analyze:

- geography
- population
- travel
- agriculture
- logistics
- political structures
- genealogy
- chronology

Flag potential issues.

Example:

A city of 100,000 in this region may require more food supply than currently modeled.

Allow the writer to ignore it.

Fantasy always takes precedence over realism.

74. World dashboard

The home screen should summarize the current world.

Potential sections:

- world map
- current historical date
- major powers
- major houses
- current conflicts
- active wars

- unresolved succession questions
- major trade routes
- recent events
- worldbuilding warnings
- manuscript progress
- recently edited entities

75. Entity pages

Every entity page should have:

- identity/header
- summary
- relationships
- timeline
- map location
- related events
- notes
- tags
- linked manuscript scenes

Contextual sections should depend on type.

A river should not look like a character page.

76. Contextual side panel

Wherever the user is working, allow quick inspection without leaving context.

Example:

Click House Marr in a scene.

Side panel:

- head
- seat
- liege
- allies
- enemies
- local territory
- relationship to scene participants

77. Bidirectional linking

If House Veyne controls Greyhaven:

House Veyne should show Greyhaven.

Greyhaven should show House Veyne.

Avoid duplicate manual entry.

78. Bulk editing

Necessary for large worlds.

Examples:

- assign several villages to one county
- change ruler for multiple holdings
- tag all settlements in region
- alter political control after conquest

79. Event-driven world changes

Events should optionally alter world state.

Example:

Conquest of Greyhaven:

Before:

Legal ruler: House Marr

After:

Military controller: House Orren

Potential later treaty:

Legal ruler becomes House Orren.

Changes should be temporally recorded rather than overwriting history.

80. Historical snapshots

Allow named snapshots.

Examples:

Before the Red War

After the Red War

At beginning of Novel One

Current manuscript date

Compare snapshots visually.

81. Difference mode

Compare two dates.

Example:

Year 220 vs Year 250.

Show:

- border changes
- rulers changed
- houses extinct
- alliances changed
- settlements founded
- roads built
- demographic changes
- trade changes

82. Character-state timeline

Character attributes may change over time.

Examples:

- title

- location
- allegiance
- marital status
- religion
- beliefs
- injuries
- wealth
- military rank

83. Scene-state snapshots

For each scene, freeze relevant world state.

The user should be able to inspect:

What was true at the exact moment this scene occurred?

84. Dependency graph

Allow entities to depend upon others.

Examples:

City depends on river.

House depends on mine revenue.

Army depends on road.

Port depends on timber supply.

Religion depends on pilgrimage route.

Use this for systemic reasoning.

85. Failure analysis

Allow the writer to select something and simulate removal/disruption.

Examples:

- bridge destroyed
- harvest fails
- ruler dies
- road blocked
- house rebels
- port blockaded
- mine exhausted

Show immediate dependencies.

86. Economics and politics interaction

Economic resources should matter politically.

Examples:

House strength may derive from:

- land
- population
- taxes
- mines
- tolls
- ports
- trade routes
- military obligations

The application should help answer:

Why is this house powerful?

rather than requiring “powerful” to be an arbitrary label.

87. Social hierarchy

Support class structures such as:

- royal
- noble
- clergy
- merchant
- artisan

- peasant
- enslaved population
- military caste

Allow custom social systems.

88. Demographics

Optional demographic modeling:

- population
- age distribution
- urban/rural ratio
- culture
- religion
- language
- social class

Keep it lightweight unless advanced modeling is enabled.

89. Disease, famine, and disasters

Historical events may affect:

- population
- trade
- agriculture
- politics
- settlement growth
- migration

Do not require detailed epidemiological simulation.

90. Migration

Cultures and populations should be capable of moving.

Track:

- origin
- destination

- period
- cause
- scale

Allow map visualization.

91. Historical borders

Political geography must support changing boundaries.

Avoid simply overwriting current regions.

92. Uncertain geography

Fantasy maps may be intentionally incomplete.

Allow:

- approximate location
- unknown boundaries
- disputed borders
- unexplored regions
- unreliable maps

93. Fog-of-knowledge maps

Optionally represent what a character or culture knows geographically.

Example:

Objective map

versus

Mara’s understanding of the world.

94. Perspective modes

Allow the world to be viewed from a selected perspective.

Examples:

Objective

House Veyne

Character Mara

Northern Church

Merchant Guild

Different perspectives may affect:

- known information
- political labels
- territorial claims
- historical interpretation
- geography knowledge

95. Narrative conflict detection

Analyze selected characters and identify conflicts between their goals.

Example:

Mara wants Oren crowned.

Edric wants Oren disinherited.

Tomas wants to avoid civil war.

Sera wants the legitimacy question suppressed.

Display intersecting motives.

96. Scene tension map

For a selected scene, display:

- alliances
- hidden relationships
- secrets

- conflicting goals
- leverage
- threats
- debts
- unresolved history

This should help a writer understand why a conversation is tense.

97. Plot consequences

Allow events in the story to feed back into world state.

Scene:

Mara reveals Oren's illegitimacy.

Consequences may update:

- character knowledge
- political factions
- succession
- relationships
- future events

The software should suggest possible affected systems but never automatically rewrite canon without approval.

98. Narrative causality graph

Plot events should connect to world consequences.

Example:

Secret revealed
→ claimant loses legitimacy
→ faction defects
→ army switches allegiance
→ city revolts

Visualize these chains.

99. Scalability

Design for worlds ranging from:

- 20 entities

to potentially:

- tens of thousands of entities
- hundreds of thousands of relationships

Dense visualizations must remain usable.

Use:

- filtering
- clustering
- virtualization
- search
- progressive rendering
- lazy loading

100. Performance requirements

Establish measurable performance expectations for:

- initial load
- search
- relationship graph
- timeline filtering
- map rendering
- world-state date changes
- large genealogy trees
- route calculation
- continuity checks

101. Data integrity

Prevent accidental corruption.

Use:

- relational constraints where appropriate
- validation
- versioning
- backups
- migration safety
- stable identifiers

102. Plugin/extensibility architecture

Consider an extension system so new modules can be added later.

Potential future modules:

- detailed economics
- warfare simulator
- heraldry designer
- language builder
- weather generation
- celestial systems
- genealogy imports
- manuscript integrations

The core should remain stable.

103. AI-assisted features

AI should be optional and never become the source of truth without user approval.

Potential AI capabilities:

- summarize an entity
- identify missing worldbuilding
- suggest consequences
- detect contradictions
- suggest scene tensions
- answer natural-language world queries
- explain dependencies
- generate brainstorming prompts

The AI should retrieve structured world facts rather than hallucinating new canon.

Clearly distinguish:

- canonical fact
- inference
- suggestion
- generated possibility

104. Canon management

Every piece of information should optionally have status:

- canon
- draft
- speculative
- deprecated
- alternate timeline
- rumor
- disputed

105. Alternate timelines

Allow branches.

Example:

Main Canon

Alternative: Oren dies at Red Ford

Explore consequences independently.

106. UX principles

Follow these principles:

1. Do not make users enter the same fact twice.
2. Every relationship should update both sides.
3. Timeline-aware information should not overwrite history.
4. Important connections should be visually inspectable.
5. Complexity should be progressive.
6. Users should always understand why derived conclusions appear.
7. The tool should facilitate writing rather than become homework.
8. Fiction takes precedence over realism.

9. Custom worlds must not be forced into medieval-European assumptions.
10. The world model should be queryable.

107. Example end-to-end workflow

Demonstrate how a writer might:

1. Create a kingdom.
2. Draw/import its map.
3. Define regions.
4. Add rivers and roads.
5. Create towns.
6. Assign towns to regional lords.
7. Create noble houses.
8. Create characters and genealogy.
9. Establish feudal obligations.
10. Define agricultural and mineral resources.
11. Create trade routes.
12. Add historical events.
13. Establish title succession.
14. Create a novel.
15. Create a scene.
16. See scene-relevant relationships and secrets.
17. Trigger a king's death.
18. Calculate succession.
19. Explore political consequences.
20. Run continuity checks.

Show how data entered earlier automatically becomes useful later.

108. MVP definition

Propose a genuinely achievable MVP.

The MVP should probably center around:

- generic entity system
- relationships
- people
- houses
- genealogy
- regions

- settlements
- map
- timeline
- basic political ownership
- search
- graph visualization

Do not attempt to implement the complete simulator at once.

Explain what belongs in:

MVP

Version 1.0

Version 2.0

Long-term

109. Architecture deliverables

Produce:

1. product requirements document
2. system architecture
3. domain model
4. database schema
5. temporal data strategy
6. geospatial architecture
7. relationship graph architecture
8. frontend architecture
9. backend architecture
10. API design
11. state-management strategy
12. caching strategy
13. search architecture
14. permissions/privacy architecture
15. import/export architecture
16. extension/plugin strategy
17. offline/sync strategy
18. testing strategy
19. security model
20. deployment architecture

110. Database design deliverables

Provide detailed proposed tables/collections for major systems including:

- projects
- entities
- entity types
- entity properties
- relationship types
- relationships
- temporal facts
- people
- genealogy
- titles
- title holders
- houses
- political control
- regions
- settlements
- geographic features
- map geometries
- roads
- waterways
- resources
- production
- trade
- organizations
- religions
- cultures
- languages
- events
- event participants
- causal relationships
- knowledge states
- secrets
- scenes
- chapters
- scene participants
- custom fields
- tags
- revisions

Show primary keys, foreign keys, indexes, and important constraints.

111. API deliverables

Design representative APIs for:

- entity CRUD
- relationship management
- genealogy queries
- succession calculation
- map layers
- world state at date
- timeline events
- route calculation
- scene context
- continuity validation
- natural-language search
- dependency queries

Use concrete request and response examples.

112. Frontend deliverables

Design screens for:

- project dashboard
- map
- timeline
- character profile
- house profile
- genealogy
- region profile
- settlement profile
- political hierarchy
- relationship graph
- trade view
- event view
- scene writing context
- universal search
- query builder

Describe major components and interactions.

113. Technical stack

Recommend a production-ready technology stack.

Evaluate realistic options for:

Frontend:

- React
- TypeScript
- mapping libraries
- graph visualization
- state management

Backend:

- appropriate server framework
- database
- geospatial support
- graph querying
- search

Desktop/local-first options:

- Tauri
- Electron
- web/PWA
- hybrid alternatives

Explain tradeoffs rather than giving arbitrary selections.

114. Testing

Include:

- unit tests
- integration tests
- database tests
- temporal consistency tests
- genealogy tests
- succession tests
- geospatial tests
- route calculation tests

- UI tests
- performance tests
- migration tests
- large-world stress tests

115. Example fictional dataset

Create a small example world to demonstrate the design.

Include at minimum:

- one kingdom
- three regions
- six settlements
- three noble houses
- one royal dynasty
- eight characters
- one disputed inheritance
- two roads
- one major river
- several resources
- two trade routes
- one historical war
- one secret
- one active political crisis

Use this dataset throughout architecture examples.

116. Key product question

Continuously evaluate every feature against:

Does this help a writer understand, navigate, reason about, or write their fictional world more easily?

Avoid adding simulation purely for technical novelty.

117. Critical product philosophy

The final system should embody this principle:

The writer should never be required to hold the entire fictional world in their head.

The software should externalize:

- memory
- spatial relationships
- chronology
- genealogy
- politics
- motivation
- causality
- economics
- logistics
- continuity
- character knowledge

The application should let the writer move fluidly between:

“What exists?”

“Where is it?”

“Who controls it?”

“When was that true?”

“Who knows it?”

“Who wants it?”

“Who benefits?”

“What depends on it?”

“What happens if it changes?”

“Why does this matter to my story?”

Final task

Using everything above, produce a rigorous software/product design rather than immediately dumping implementation code.

Begin by:

1. identifying the major domain boundaries,
2. identifying areas where naive architecture would fail,
3. proposing the core data model,
4. defining the temporal and relationship architecture,
5. defining the MVP,
6. proposing the long-term architecture,
7. designing the principal interfaces and visualization modes,
8. identifying difficult technical problems,
9. proposing solutions and tradeoffs,
10. and creating a staged implementation roadmap.

Pay particular attention to avoiding architectural decisions in the MVP that would make the later advanced systems impossible.

Where there are multiple reasonable approaches, compare them explicitly.

Treat this as software intended to eventually become a serious, polished application rather than a weekend prototype.